# Computing Systems and Programming (6): gpio/binout and gpio/binin

Dr. Clifford Choy

School of Design

16 Oct 2026

# gpio/binout/ binout_timedwrite and binout_serialin_timedwrite

# gpio/binout/ binout_timedwrite

`BinOutControl` – a **class** for Binary Output
`binOutCtrl0` – a `BinOutControl` **object**

`binOutHandler` is for `BinOutControl` class

Initialize `binOutCtrl0` and request resources for using this object (Binary Output), and associate to IO12

Enable scheduler of `binOutctrl0`, so that
- `binOutHandler` is invoked every 1ms, and
- with **two** timeout events simultaneously

```
10  #include <IO_Control.h>
11
12  BinOutControl binOutCtrl0;
13  ulong_t onDurationMS = 1000;
14  ulong_t offDurationMS = 2000;
15
16  DeclareHandler(printBinOutState,_ctrl,BinOutControl,{…
22  });
23
24  DeclareHandler(changeBinOutState,_ctrl,BinOutControl,{…
32  });
33
34  DeclareHandler(binOutHandler,_ctrl,BinOutControl,{ //  declare a handler for
47  });
48
49  void setup() {
50    // put your setup code here, to run once:
51    IO_Control.begin();
52
53    Serial.begin(115200);
54    Serial.println("chip started");
55
56    binOutCtrl0.init(12,LOW);
57
58    binOutCtrl0.enableScheduler(0, 1, binOutHandler,2);
59  }
60
61  void loop() {
62    // put your main code here, to run repeatedly:
63  }
```

Invoke `changeBinOutState`, only ONCE

`changeBinOutState` is for changing state of `binOutCtrl0` each time when it is invoked

`_ctrl == binOutctrl0` when `changeBinOutState` is invoked

`_ctrl.read()` – read current state of the binary output (at IO12)

Invoke `changeBinOutState` after duration depends on current state of `binOutCtrl0`

`_ctrl.write()` – set the state of binary output (at IO12)
Can be `HIGH` or `LOW` only

Invoke `printBinOutState`, only ONCE

`printBinOutState` is for printing out current state of `binOutCtrl0` when it is invoked

```
10  #include <IO_Control.h>
11
12  BinOutControl binOutCtrl0;
13  ulong_t onDurationMS = 1000;
14  ulong_t offDurationMS = 2000;
15
16  DeclareHandler(printBinOutState,_ctrl,BinOutControl,{
17    Serial.print("current state=");
18    Serial.print(_ctrl.read());
19    Serial.print(" at t=");
20    Serial.println(_ctrl.getUpdateTime());
21    _ctrl.setTimeoutByRelTime(1,printBinOutState,500,true);
22  });
23
24  DeclareHandler(changeBinOutState,_ctrl,BinOutControl,{
25    if (_ctrl.read()==HIGH)
26    { _ctrl.write(LOW);
27      _ctrl.setTimeoutByRelTime(0,changeBinOutState,offDurationMS,true);
28    } else
29    { _ctrl.write(HIGH);
30      _ctrl.setTimeoutByRelTime(0,changeBinOutState,onDurationMS,true);
31    }
32  });
33
34  DeclareHandler(binOutHandler,_ctrl,BinOutControl,{ //  declare a handler for s
35    //  binOutHandler - name of the handler function
36    //  _ctrl - BinOutControl object
37    //  BinOutControl - class of object of this handler
38    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
39
40    // if ( ctrl.isSchedulerTimeEvent())
41    //      Serial..println("BinOutControl - SchedulerTimerEvent");
42    if (! ctrl.isTimeoutActive(0))
43      _ctrl.setTimeoutByRelTime(0,changeBinOutState,0,true);
44    if (! ctrl.isTimeoutActive(1))
45      _ctrl.setTimeoutByRelTime(1,printBinOutState,0,true);
46
47  });
```

# binout_timedwrite (1)

- `BinOutControl`
  - A **class** for a binary output (“GPIO output”)
  - String is another example of a **class**
  - Think of it as an extended data type

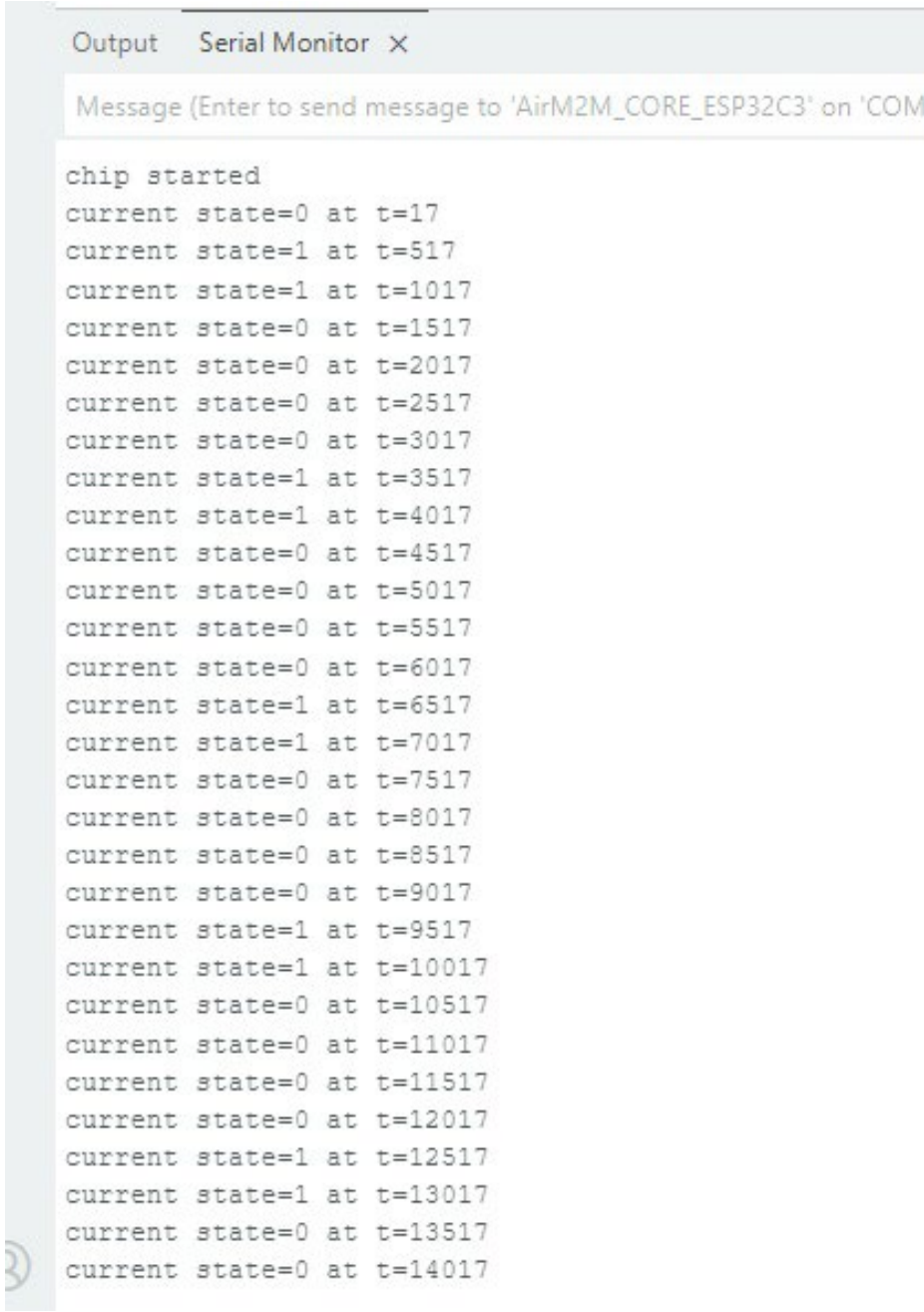
- `binOutCtrl0`
  - A variable of type (class) `BinOutControl`
  - OR, a `BinOutControl` **object**
  - OR, an **instance** of the class `BinOutControl`
  - A class can have more than one objects/instances in a given program

# binout_timedwrite (2)

- How to use a `BinOutControl` object
  1. Initialize it using `init` function
  2. Optionally use `enableScheduler` function to activate a handler function for this specific object (independent of other handler functions, e.g. the one for the object `IO_Control` in previous examples like `iocontrol/iocontrol_serialout`)
- Selected functions/methods of `BinOutControl` object
  - `read`
  - `write`
- Refer to https://www.designandmake.org/x/MWtMDg on methods of `BinOutControl`

# binout_timedwrite (3)

- Behavior of the program
  - LED at IO12 is off
  - Scheduler starts as soon as possible and invoke `changeBinOutState` first time, which repeats the following:
    - Turns on LED at IO12, and waits for duration of `onDurationMS` (1000ms)
    - Turns off LED at IO12, and waits for duration of `offDurationMS` (2000ms)



# binout_timedwrite (4)

- Note that `BinOutControl` and `IOControl` share the same base class, and hence all functions which can be used on the object `IO_Control` (instance of `IOControl`) can also be used on instance of `BinOutControl`
- Hence,
  - `enableScheduler`
  - `getUpdateTime`
  - `setTimeoutByRelTime`
  - …
- Refer to “binout_timedwrite_2leds.ino” for control of 2 leds

# gpio/binout/ binout_serialin_timedwrite

# binout_serialin_timedwrite (1)

Added a handler for `IO_Control`, for reading inputs from Serial Monitor

Refer to the example `iocontrol/iocontrol_serialin`

Handler for `IO_Control`

All other handlers are the same as those in `gpio/binout/binout_timedwrite`

```
15  #include <IO_Control.h>
16
17  BinOutControl binOutCtrl0;
18  ulong_t onDurationMS = 1000;
19  ulong_t offDurationMS = 2000;
20
21  DeclareHandler(iocontrolHandler,_ctrl,IOControl,{ ···
99  });
100
101 DeclareHandler(printBinOutState,_ctrl,BinOutControl,{···
107 });
108
109 DeclareHandler(changeBinOutState,_ctrl,BinOutControl,{···
117 });
118
119 DeclareHandler(binOutHandler,_ctrl,BinOutControl,{ //  declare a handler for schedu
132 });
133
134 void setup() {
135   // put your setup code here, to run once:
136   IO_Control.begin();
137
138   Serial.begin(115200);
139   Serial.println("chip started");
140
141   IO_Control.enableScheduler(0,500,iocontrolHandler);
142
143   binOutCtrl0.init(12,LOW);
144
145   binOutCtrl0.enableScheduler(0, 1, binOutHandler,2);
146 }
147
```

```
21  DeclareHandler(iocontrolHandler,_ctrl,IOControl,{
22    // if (_ctrl.isSchedulerTimeEvent())
23    //     Serial.println("IOControl - SchedulerTimerEvent");
24
25    if (Serial.available() > 0) {
26      String input = Serial.readStringUntil('\n');
27      Serial.print("input command:");
28      Serial.println(input);
29
30      Serial.print("current time=");
31      Serial.println(_ctrl.getUpdateTime());
32
33      int pos=input.indexOf(',');
34
35      if (pos==-1)
36      { //  no comma, assuming user enter one number, and onDurationMS is set to the number
37        //  offDurationMS is unchanged
38
39        int val = input.toInt();
40        if (val<=0)
41        { Serial.println("Error: invalid duration; onDurationMS and offDurationMS unchange");
42          return;
43        }
44        onDurationMS = val;
45        Serial.print("Duration for ON=");
46        Serial.println(onDurationMS);
47        Serial.print("Duration for OFF=");
48        Serial.println(offDurationMS);
49
50        return;
51      }
52
53      if (pos==0)
54      { //  comma at beginning, then no change to onDurationMS and only change offDurationMS
55
56        int val = input.substring(1).toInt();
57        if (val<=0)
58        { Serial.println("Error: invalid duration for offDurationMS; onDurationMS and offDurationMS unchange");
59          return;
60        }
61        offDurationMS = val;
62
63        Serial.print("Duration for ON=");
64        Serial.println(onDurationMS);
65        Serial.print("Duration for OFF=");
66        Serial.println(offDurationMS);
67
68        return;
69      }
71      if (pos>0)
72      { //  at least one comma, assuming the form "<number1>,<number2>"
73        if (input.indexOf(',',pos+1) !=-1)
74        { //  more than one comma, and hence error
75          Serial.println("Error: more than one comma; onDurationMS and offDurationMS unchange");
76          return;
77        }
78        int _onDurationMS = input.substring(0,pos).toInt();
79        int _offDurationMS = input.substring(pos+1).toInt();
80        if (_onDurationMS<=0)
81        { Serial.print("Error: ON duration <=0 (");
82          Serial.print(_onDurationMS);
83          Serial.println("); onDurationMS and offDurationMS unchange");
84          return;
85        }
86        if (_offDurationMS<=0)
87        { Serial.print("Error: ON duration <= 0 (");
88          Serial.print(_offDurationMS);
89          Serial.println("); onDurationMS and offDurationMS unchange");
90          return;
91        }
92        onDurationMS = _onDurationMS;
93        offDurationMS = _offDurationMS;
94        Serial.print("Duration for ON=");
95        Serial.println(onDurationMS);
96        Serial.print("Duration for OFF=");
97        Serial.println(offDurationMS);
98      }
99    }
```

For the case *<onDurationMS>*

For the case *,<offDurationMS>*

For the case *<onDuratioS>,<offDurationMS>*

# binout_serialin_timedwrite (2)

**Input:** 500

```
chip started
current state=0 at t=19
current state=1 at t=519
current state=1 at t=1019
current state=0 at t=1519
current state=0 at t=2020
current state=0 at t=2520
current state=0 at t=3020
current state=1 at t=3520
input command:500
current time=4001
Duration for ON=500
Duration for OFF=2000
current state=1 at t=5003
current state=0 at t=5503
current state=0 at t=6003
current state=0 at t=6504
current state=0 at t=7004
current state=1 at t=7504
current state=0 at t=8004
current state=0 at t=8505
```

**Input:** ,4000

```
current state=0 at t=19006
current state=0 at t=19506
input command:,4000
current time=20001
Duration for ON=500
Duration for OFF=4000
current state=1 at t=21003
current state=0 at t=21503
current state=0 at t=22003
current state=0 at t=22504
current state=0 at t=23004
current state=0 at t=23504
current state=0 at t=24004
current state=0 at t=24505
current state=1 at t=25005
current state=0 at t=25505
current state=0 at t=26005
current state=0 at t=26506
```

**Input:** 2000,500

```
current state=0 at t=27506
current state=0 at t=28006
input command:2000,500
current time=28500
Duration for ON=2000
Duration for OFF=500
current state=0 at t=29501
current state=1 at t=30001
current state=1 at t=30502
current state=1 at t=31002
current state=1 at t=31502
current state=0 at t=32002
current state=1 at t=32503
current state=1 at t=33003
current state=1 at t=33503
current state=1 at t=34003
current state=0 at t=34504
current state=1 at t=35004
current state=1 at t=35504
current state=1 at t=36004
current state=1 at t=36505
current state=0 at t=37005
current state=1 at t=37505
current state=1 at t=38005
```

# binout_serialin_timedwrite (3)

- Added `iocontrolHandler` to process inputs from Serial Monitor
  - Refer to `iocontrol/iocontrol_serialin`
- Note:
  - each object can have its own handler
  - `IO_Control` is an object

- Functions for String
  - `indexOf(',')`
  - `indexOf(',',pos)`
  - `substring(pos1,pos2)`
  - `toInt()`

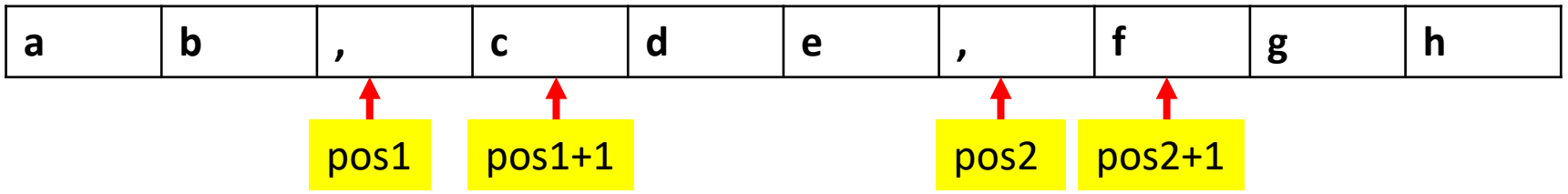
# `indexOf()` and `substring()` (1)

- Refer to Example6_1.zip

```
10
11      // assuming "input" has two commas
12      int pos1=input.indexOf(",");
13      int pos2=input.indexOf(",",pos1+1);
14      Serial.print("pos1=");
15      Serial.println(pos1);
16      Serial.print("pos2=");
17      Serial.println(pos2);
18
19      Serial.print("first=");
20      Serial.println(input.substring(0,pos1));  // print part of string before first comma
21      Serial.print("second=");
22      Serial.println(input.substring(pos1+1,pos2)); // print part of string between first and second comma
23      Serial.print("third=");
24      Serial.println(input.substring(pos2+1));  // print part of string after second comma
25  }
```

# `indexOf()` and `substring()` (2)

- Value of `input`




- `pos1 = input.indexOf(",")`
  - Value of `pos1` is `2`
- Value of `pos1 + 1` is `3`
- `pos2 = input.indexOf(",",pos1+1)`
  - Value of `pos2` is `6`

# `indexOf()` and `substring()` (3)

- `input.substring(0,pos1)`
  - Return a value of `"ab"`
- `input.substring(pos1+1,pos2)`
  - Return a value of `"cde"`
- `input.substring(pos2+1)`
  - Return a value of `"fgh"`

# Task 6 (1)

- Add the heading “Task 6” in your submission page

- Extend “gpio/binout/binout_serialin_timedwrite” with the following features:
  - Support blinking of two LEDs, one at IO12 and another at IO13
  - Allow users to input commands to control blinking of each LED:
    - `<led_index>,<on_duration>, <off_duration>, <no_of_times>` OR
    - `<led_index>,<relative_start_time>, <on_duration>, <off_duration>, <no_of_times>`
  - `<led_index> == 0` for IO12 and `<led_index> == 1` for IO13
- Example 1:
  - User input: `0,1000,500,10`
  - LED at IO12 will be turned on for 1000ms and turned off for 500ms, and repeat for 10 times
- Example 2:
  - User input: `1,500,1000,0`
  - LED at IO13 will be turned on for 500ms and turned off for 1000ms, and repeat until another command for IO13 is provided
- Example 3:
  - User input: `0,1000,1000,500,10`
  - LED at IO12 will be turned on for 1000ms and turned off for 500ms, and repeat for 10 times. The whole sequence of on/off will be started 1000ms after the current time.

# Task 6 (2)

- Compile and upload your program to ESP32C3
- Video record the output from Serial Monitor to demonstrate the above logic for selected test cases, one video per test case (at least using the above three examples), and upload to the submission page. Be reminded to clearly state what is the input for the corresponding video.
- Save and compress your sketch (in zip format), and upload to the submission page

# Task 6 (3)

- Hints:
  - Study Example6_2.zip and modify from the example
  - You will have to consider the following:
    - How to handle one more LED (at IO13)?
    - How to modify the logic for parsing input command?
    - Do you want input command to have no effect while blinking? Or just override the current setting when a new input command is entered while blinking?

# BinOutSequence

# `BinOutSequence`

- A class for a sequence of commands for sending to `BinOutControl` object
- Examples:
  - gpio/binout/binout_play_oneseq
  - gpio/binout/binout_play_altseq
  - gpio/binout/binout_serialin_playseq_immed
  - gpio/binout/binout_serialin_playseq_follow

- Documentations
  - https://www.designandmake.org/x/MWtMDg (for `BinOutControl`)
  - https://www.designandmake.org/x/MmtMDg (for `BinOutSequence`)

# gpio/binout/
# binout_play_oneseq

```
#include <IO_Control.h>

BinOutControl binOutCtrl0;
BinOutSequence binOutSeq0;

DeclareHandler(binOutHandler,_ctrl,BinOutControl,{
  //  binOutHandler - name of the handler functior
  //  _ctrl - BinOutControl object
  //  BinOutControl - class of object of this hanc
  //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked

  // if (_ctrl.isSchedulerTimeEvent(_status))
  //   Serial.println("BinOutControl - SchedulerTimerEvent");
  if (_ctrl.isAllSeqStarted())
  { Serial.print("BinOutControl - AllSeqStarted, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
  if (_ctrl.isAllSeqEnded())
  { Serial.print("BinOutControl - AllSeqEnded, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
  if (_ctrl.isOneSeqStarted())
  { Serial.print("BinOutControl - OneSeqStarted, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
  if (_ctrl.isOneSeqNewCmdStarted())
  { Serial.print("BinOutControl - OneSeqNewCmdStarted, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
  if (_ctrl.isOneSeqPaused())
  { Serial.print("BinOutControl - OneSeqPaused, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
  if (_ctrl.isOneSeqEnded())
  { Serial.print("BinOutControl - OneSeqEnded, at time=");
    Serial.println(_ctrl.getUpdateTime());
  }
});
```

a `BinOutSequence` object

Functions for reporting status while playing a sequence

```
void setup() {
  // put your setup code here, to run once:
  IO_Control.begin();

  Serial.begin(115200);
  Serial.println("chip started");

  binOutCtrl0.init(12,LOW);

  binOutSeq0.addSetState(500, HIGH);
  binOutSeq0.addSetState(500, LOW);
  binOutSeq0.addSetState(500, HIGH);
  binOutSeq0.addSetState(500, LOW);
  binOutSeq0.addSetState(500, HIGH);
  binOutSeq0.addSetState(500, LOW);

  binOutCtrl0.enableScheduler(1000, 1, binOutHandler);
  binOutCtrl0.playRelTime(binOutSeq0, 2000, 2);
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

IO12 is initially LOW

Add **commands** to `binOutSeq0`

Start playing the sequence at time = 2000ms **relative** to the current time, for 2 times

Note differences of this program as compared to “binout_timedwrite”

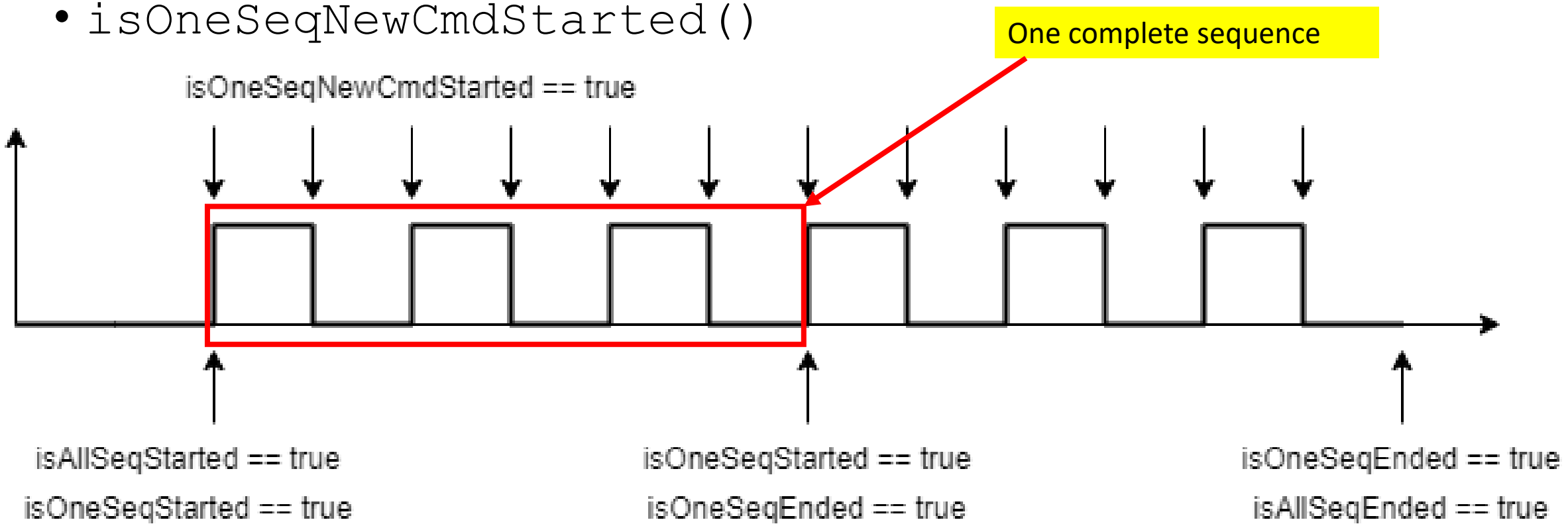
# binout_play_oneseq (1)

- `BinOutSequence`
  - `AddSetState` – add a command to the sequence
  - e.g. `AddSetState(500,HIGH)` means “turn HIGH and then stay for 500ms”

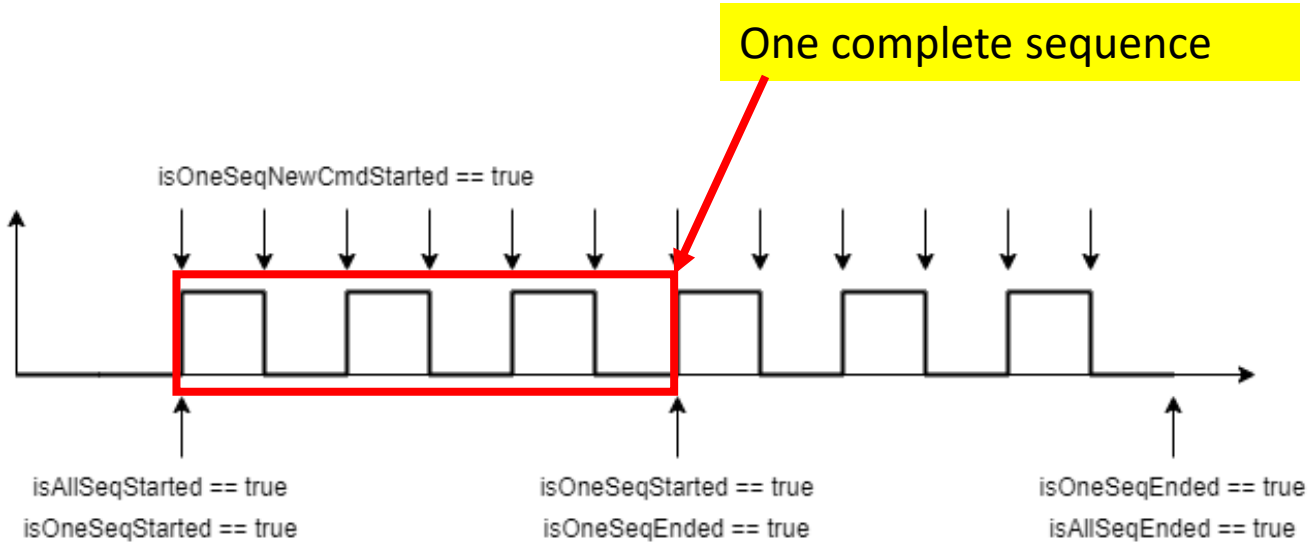
- `playRelTime(seq,start_time,num)`
  - Play a `BinOutSequence` object (`seq`) starting from a given time (`start_time`) **relative to current time**, for a given number of times (`num`)
  - `start_time` == 0 means “start immediately”

# binout_play_oneseq (2)

- `isAllSeqStarted(),isAllSeqEnded()`
- `isOneSeqStarted(),isOneSeqEnded()`
- `isOneSeqNewCmdStarted()`



# binout_play_oneseq (3)




```
chip started
BinOutControl - AllSeqStarted, at time=2016
BinOutControl - OneSeqStarted, at time=2016
BinOutControl - OneSeqNewCmdStarted, at time=2016
BinOutControl - OneSeqNewCmdStarted, at time=2517
BinOutControl - OneSeqNewCmdStarted, at time=3018
BinOutControl - OneSeqNewCmdStarted, at time=3519
BinOutControl - OneSeqNewCmdStarted, at time=4020
BinOutControl - OneSeqNewCmdStarted, at time=4521
BinOutControl - OneSeqEnded, at time=5021
BinOutControl - OneSeqStarted, at time=5022
BinOutControl - OneSeqNewCmdStarted, at time=5022
BinOutControl - OneSeqNewCmdStarted, at time=5523
BinOutControl - OneSeqNewCmdStarted, at time=6024
BinOutControl - OneSeqNewCmdStarted, at time=6525
BinOutControl - OneSeqNewCmdStarted, at time=7026
BinOutControl - OneSeqNewCmdStarted, at time=7527
BinOutControl - AllSeqEnded, at time=8027
BinOutControl - OneSeqEnded, at time=8027
```

# gpio/binout/ binout_play_altseq

```
11  BinOutControl binOutCtrl0;
12  BinOutSequence binOutSeq0;
13  BinOutSequence binOutSeq1;
14  int binOutState = 0;
```

two `BinOutSequence` objects

Start playing another sequence immediately for one time when the current one ends

```
44    if (_ctrl.isAllSeqEnded()) {
45      switch (binOutState) {
46        case 0:
47          Serial.println("play binOutSeq1");
48          _ctrl.playRelTime(binOutSeq1, 0, 1);
49          binOutState=1;
50          break;
51        case 1:
52          Serial.println("play binOutSeq0");
53          _ctrl.playRelTime(binOutSeq0, 0, 1);
54          binOutState=0;
55          break;
56        default:
57          break;
58      }
59    }
```

Corresponds to Morse code of “O”

```
71    binOutSeq0.addSetState(500, HIGH);
72    binOutSeq0.addSetState(500, LOW);
73    binOutSeq0.addSetState(500, HIGH);
74    binOutSeq0.addSetState(500, LOW);
75    binOutSeq0.addSetState(500, HIGH
76    binOutSeq0.addSetState(500, LOW)
77
```

Corresponds to Morse code of “S”

```
78    binOutSeq1.addSetState(1000, HIGH);
79    binOutSeq1.addSetState(500, LOW);
80    binOutSeq1.addSetState(1000, HIGH);
81    binOutSeq1.addSetState(500, LOW);
82    binOutSeq1.addSetState(1000, HIGH);
83    binOutSeq1.addSetState(500, LOW);
84
85    binOutCtrl0.enableScheduler(1000, 1, binOutHandler);
86    binOutCtrl0.playRelTime(binOutSeq0, 2000, 1);
```

# binout_play_altseq (1)

- `playRelTime(…)` can be invoked while inside handler
- Refer to “iocontrol_serialout_fsm1” on binOutState
- Alternatively playing `binOutSeq0` and `binOutSeq1`, one immediately after previous one

```
chip started
BinOutControl - AllSeqStarted, at time=2016
BinOutControl - AllSeqEnded, at time=5021
play binOutSeq1
BinOutControl - AllSeqStarted, at time=5022
BinOutControl - AllSeqEnded, at time=9527
play binOutSeq0
BinOutControl - AllSeqStarted, at time=9528
BinOutControl - AllSeqEnded, at time=12533
play binOutSeq1
BinOutControl - AllSeqStarted, at time=12534
BinOutControl - AllSeqEnded, at time=17039
play binOutSeq0
BinOutControl - AllSeqStarted, at time=17040
BinOutControl - AllSeqEnded, at time=20045
play binOutSeq1
BinOutControl - AllSeqStarted, at time=20046
BinOutControl - AllSeqEnded, at time=24551
play binOutSeq0
BinOutControl - AllSeqStarted, at time=24552
BinOutControl - AllSeqEnded, at time=27557
play binOutSeq1
BinOutControl - AllSeqStarted, at time=27558
BinOutControl - AllSeqEnded, at time=32063
```

# gpio/binout/ binout_serialin_playseq_immed

Refer to
iocontrol/iocontrol_serialin

```
11  BinOutControl binOutCtrl0;
12  BinOutSequence binOutSeq0;
13  BinOutSequence binOutSeq1;
14
15  DeclareHandler(iocontrolHandler,_ctrl,IOC
16    // if (_ctrl.isSchedulerTimeEvent())
17    //     Serial..println("IOControl - SchedulerTimerEvent");
18    if (Serial.available() > 0) {
19      String input = Serial.readStringUntil('\n');
20      Serial.print("input:");
21      Serial.println(input);
22
23      Serial.print("current time=");
24      Serial.println(_ctrl.getUpdateTime());
25
26      if (input.equals("0"))
27      { binOutCtrl0.playRelTime(binOutSeq0, 0, 1);
28        Serial.println("start playing sequence 0");
29        return;
30      }
31      if (input.equals("1"))
32      { binOutCtrl0.playRelTime(binOutSeq1, 0, 1);
33        Serial.println("start playing sequence 1");
34        return;
35      }
36      Serial.println("unknown request");
37
38      //  same as above if statements, but using if ... else ... structure
39      /*
40        if (input.equals("0"))
41        { binOutCtrl0.playRelTime(binOutSeq0, 0, 1);
42          Serial.println("start playing sequence 0");
43        } else
44        { if (input.equals("1"))
45          { binOutCtrl0.playRelTime(binOutSeq1, 0, 1);
46            Serial.println("start playing sequence 1");
47          } else
48          { Serial.println("unknown request");
49          }
50        }
51      */
52
53    }
54  });
```

Same logic

```
57  DeclareHandler(binOutHandler,_ctrl,BinOutControl,{
58    // if (_ctrl.isSchedulerTimeEvent())
59    //   Serial.println("BinOutControl - SchedulerTimerEvent");
60    if (_ctrl.isAllSeqStarted())
61    { Serial.print("BinOutControl - AllSeqStarted, at time=");
62      Serial.println(_ctrl.getUpdateTime());
63    }
64    if (_ctrl.isAllSeqEnded())
65    { Serial.print("BinOutControl - AllSeqEnded, at time=");
66      Serial.println(_ctrl.getUpdateTime());
67    }
68    // if (_ctrl.isOneSeqStarted())
69    // { Serial.print("BinOutControl - OneSeqStarted, at time=");
70    //   Serial.println(_currentMillis);
71    // }
72    // if (_ctrl.isOneSeqNewCmdStarted())
73    // { Serial.print("BinOutControl - OneSeqNewCmdStarted, at time=");
74    //   Serial.println(_currentMillis);
75    // }
76    // if (_ctrl.isOneSeqPaused())
77    // { Serial.print("BinOutControl - OneSeqPaused, at time=");
78    //   Serial.println(_currentMillis);
79    // }
80    // if (_ctrl.isOneSeqEnded())
81    // { Serial.print("BinOutControl - OneSeqEnded, at time=");
82    //   Serial.println(_currentMillis);
83    // }
84
85  });
```

# binout_serialin_playseq_immed (1)

Sequence 0 is played without interrupt

Sequence 1 is started playing

Playing of sequence 1 is interrupted by sequence 0

```
chip started
input:0
current time=8501
start playing sequence 0
BinOutControl - AllSeqStarted, at time=9504
BinOutControl - AllSeqEnded, at time=12516
input:1
current time=16301
start playing sequence 1
BinOutControl - AllSeqStarted, at time=17304
input:0
current time=18200
start playing sequence 0
BinOutControl - AllSeqEnded, at time=21714
```

# binout_serialin_playseq_immed (2)

- `playRelTime(…)` can be invoked while an existing sequence is being played by `binOutCtrl0`
- Note that in the above situation, playing of existing sequence will be stopped and new sequence will be played back immediately

# gpio/binout/ binout_serialin_playseq_follow

```
11  BinOutControl binOutCtrl0;
12  BinOutSequence binOutSeq0;
13  BinOutSequence binOutSeq1;
14  int nextSeqIndex = -1;
15
16  DeclareHandler(iocontrolHandler,_ctrl
17    // if (_ctrl.isSchedulerTimeEvent())
18    //   Serial.println("IOControl - Sch
19
20    if (Serial.available() > 0) {
21      String input = Serial.readStringUntil('\n');
22      Serial.print("input:");
23      Serial.println(input);
24
25      Serial.print("current time=");
26      Serial.println(_ctrl.getUpdateTime());
27
28      if (input.equals("0"))
29      { if (binOutCtrl0.isPlaying())
30        { nextSeqIndex=0;
31          Serial.println("going to play sequence 0 after current one finished");
32        } else
33        { nextSeqIndex=-1;
34          binOutCtrl0.playRelTime(binOutSeq0, 0, 1);
35          Serial.println("start playing sequence 0 now");
36        }
37        return;
38      }
39      if (input.equals("1"))
40      { if (binOutCtrl0.isPlaying())
41        { nextSeqIndex=1;
42          Serial.println("going to play sequence 1 after current one finished");
43        } else
44        { nextSeqIndex=-1;
45          binOutCtrl0.playRelTime(binOutSeq1, 0, 1);
46          Serial.println("start playing sequence 1 now");
47        }
48        return;
49      }
50      Serial.println("unknown request");
```

If `nextSeqIndex != -1`, value of `nextSeqIndex` corresponds to the sequence to be played next after the current one ended

true if `binOutCtrl0` is playing a sequence

Case when a sequence is currently being played

Case when no sequence is currently being played

Code fragment in `binOutHandler`

```
102   if (_ctrl.isAllSeqEnded())
103   { if (nextSeqIndex!=-1)
104     { if (nextSeqIndex==0)
105         _ctrl.playRelTime(binOutSeq0, 0, 1);
106       else if (nextSeqIndex==1)
107                 _ctrl.playRelTime(binOutSeq1, 0, 1);
108       Serial.print("start playing sequence ");
109       Serial.print(nextSeqIndex);
110       Serial.println(" now");
111       nextSeqIndex=-1;
112     }
113   }
```

Play the sequence which was being requested to be played (as indicated in `nextSeqIndex`) while the previous sequence was being played

# binout_serialin_playseq_follow (1)

Sequence 0 is the first sequence to be played when chip started

Sequence 0 is still playing

Sequence 0 just finished playing

Playing of sequence 1 started after sequence 0 is finished playing

```
chip started
start playing sequence 0 now
BinOutControl - AllSeqStarted, at time=2016
BinOutControl - AllSeqEnded, at time=5027
input:0
current time=11621
start playing sequence 0 now
BinOutControl - AllSeqStarted, at time=12623
input:1
current time=13410
going to play sequence 1 after current one finished
BinOutControl - AllSeqEnded, at time=16421
start playing sequence 1 now
BinOutControl - AllSeqStarted, at time=16423
BinOutControl - AllSeqEnded, at time=20935
```

# binout_serialin_playseq_follow (2)

- Added logic to make sure if a sequence is currently being played, the new sequence will be played after the current one
- `nextSeqIndex`
  - A variable to keep track of which sequence to be played next when a sequence is already being played

# Task 7 (1)

- Add the heading “Task 7” in your submission page

- By modifying the example “gpio/binout/binout_play_altseq”, write a program which allows user to input command through Serial Monitor to control which sequence to play for the LED at IO12. The input command is in the form:
  - *<sequence no #1>*, *<sequence no #2>*, *<sequence no #3>*, *<sequence no #4>*
  - where *<sequence no #1>* is a number corresponding to the sequence to be played, and similarly for *<sequence no #2>*, <sequence no #3> and *<sequence no #4>*

# Task 7 (2)

- Example 1:
  - User input: 0,2,1,2
  - Play the sequences in the order **binOutSeq0**, **binOutSeq2**, **binOutSeq1** and **binOutSeq2**
  - Note: “0” corresponds to **binOutSeq0**, “1” corresponds to **binOutSeq1**, and “2” corresponds to **binOutSeq2**
- Example 2:
  - User input: 1,2,-1,0
  - Play the sequences in the order **binOutSeq1** and **binOutSeq2**, and then wait for 2000ms with LED at IO12 is OFF, and then play the sequence **binOutSeq0**
  - Note that “-1” means “turn off for 2000ms”

# Task 7 (3)

- Note that this program only supports three pre-defined sequences (that is, 0, 1 and 2), as well as the value of “-1”:
  - “-1” corresponds to “turn off for 200ms”
  - “0” corresponds to the Morse code of “X”
  - “1” corresponds to the Morse code of “Y”
  - “2” corresponds to the Morse code of “Z”

- Compile and upload your program to ESP32C3
- Video record the output from Serial Monitor to demonstrate the above logic for selected test cases, one video per test case (at least using the above two examples), and upload to the submission page. Be reminded to clearly state what is the input for the corresponding video.
- Save and compress your sketch (in zip format), and upload to the submission page

# gpio/binin/
# binin_changetime

A `BinInControl` object for GPIO input

true if GPIO input at IO09 has changed state
i.e. from HIGH to LOW, or LOW to HIGH

For getting time of change, in milliseconds

For getting state AFTER the change

```
8   BinInControl binInCtrl0;
9
10  DeclareHandler(binInHandler,_ctrl,BinInControl,{
11    // if (_ctrl.isSchedulerTimeEvent(_status))
12    //   Serial.println("BinOutControl - SchedulerTimerEvent");
13    if (_ctrl.isBinInTriggered())
14    { Serial.print("event time=");
15      Serial.print(_ctrl.getInterruptTime());
16      if (_ctrl.getInterruptState())
17        Serial.println(" from LOW to HIGH");
18      else
19        Serial.println(" from HIGH to LOW");
20    }
21  });
```

```
23  void setup() {
24    // put your setup code here, to run once:
25    IO_Control.begin();
26
27    Serial.begin(115200);
28    Serial.println("chip started");
29
30    binInCtrl0.init(9,INPUT);
31
32    int button_state = binInCtrl0.read();
33    Serial.print("IO09 state=");
34    Serial.println(button_state);
35
36    binInCtrl0.enableScheduler(500, 10, binInHandler);
37    binInCtrl0.enableInterrupt(CHANGE);
38  }
```

IO09 is initially HIGH

Use IO09 as GPIO input

Read current state of IO09

Enable detection of change in state of IO09

Output Serial M

When button is pressed

Message (Enter to send message to 'AirM2M_

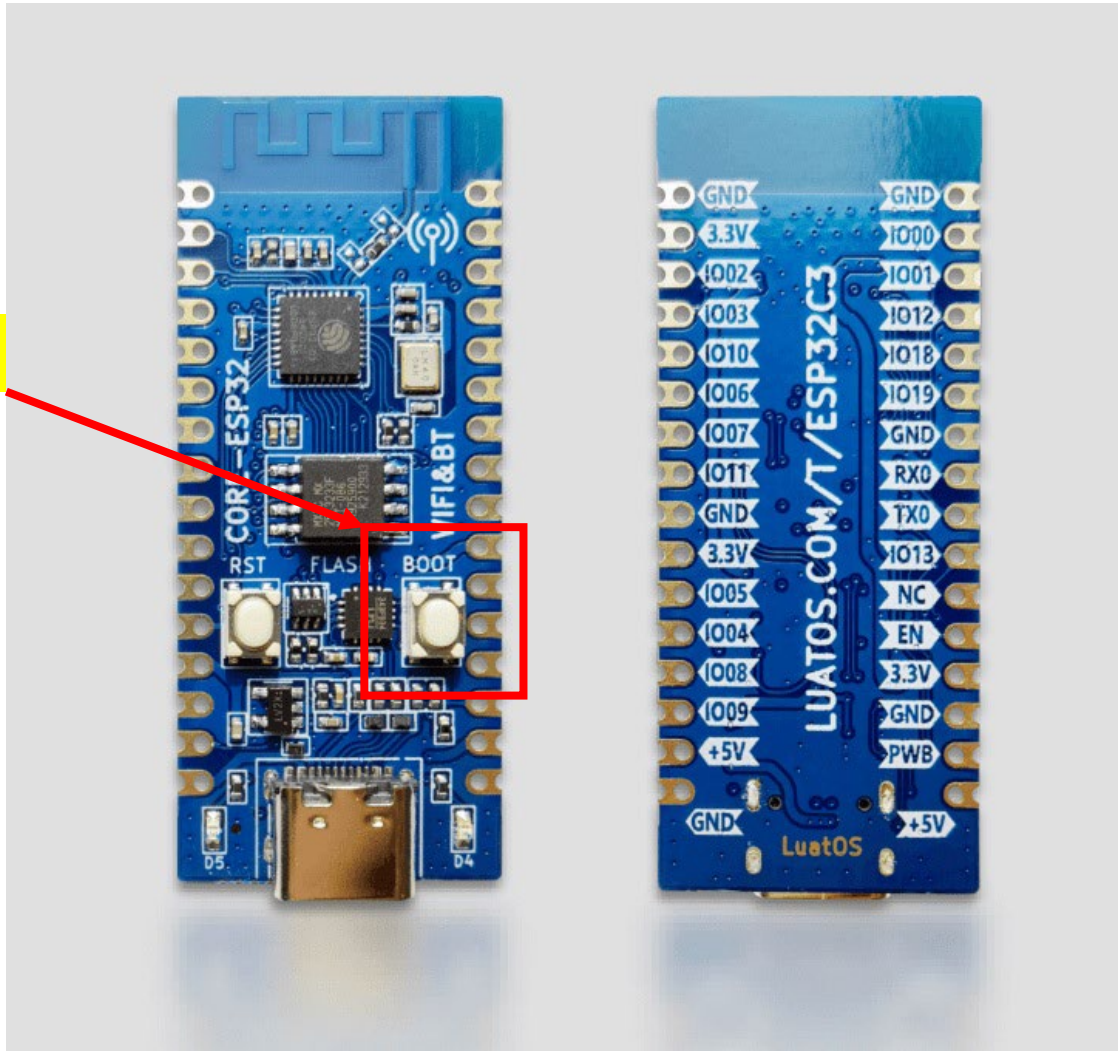
```
chip started
IO09 state=1
event time=27273 from HIGH to LOW
event time=27458 from LOW to HIGH
event time=28861 from HIGH to LOW
event time=31124 from LOW to HIGH
event time=31895 from HIGH to LOW
event time=32695 from LOW to HIGH
event time=33796 from HIGH to LOW
event time=34196 from LOW to HIGH
```

When button is released

# binin_changetime (1)

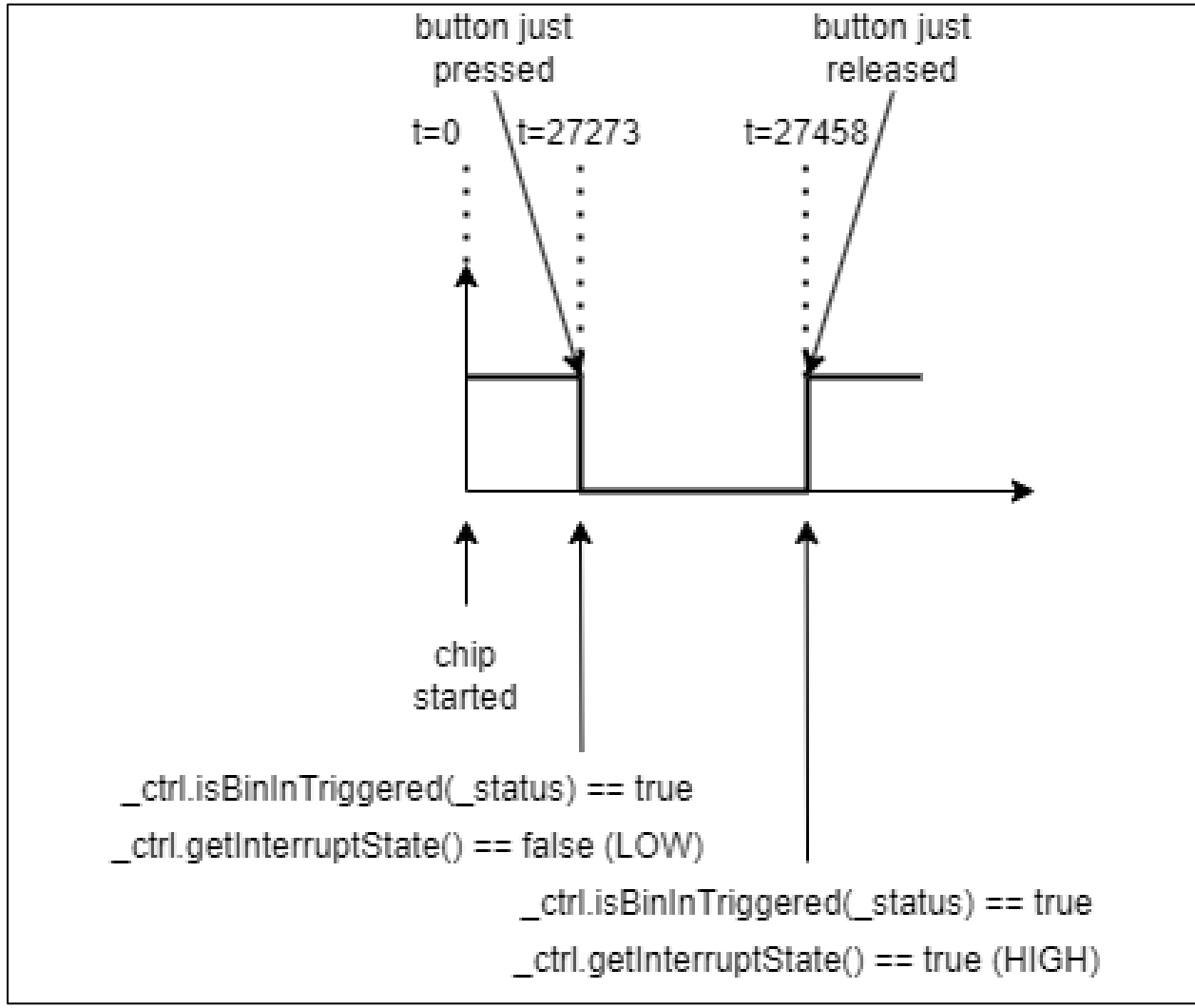
- Documentations:
  - https://www.designandmake.org/x/M2tMDg

- IO09 is the “BOOT” button
- `HIGH` if NOT pressed
- `LOW` if pressed



# binin_changetime (2)

- `isBinInTriggered()`
- `getInterruptState()`
  - State of the GPIO output AFTER the change
- `getInterruptTime()`
  - Time when the state change occurred




Output Serial Monitor ×

Message (Enter to send message to 'AirM2M_COR

```
chip started
IO09 state=1
event time=27273 from HIGH to LOW
event time=27458 from LOW to HIGH
event time=28861 from HIGH to LOW
event time=31124 from LOW to HIGH
event time=31895 from HIGH to LOW
event time=32695 from LOW to HIGH
event time=33796 from HIGH to LOW
event time=34196 from LOW to HIGH
```

# Task 8

- Add the heading “Task 8” in your submission page

- Revise the example “gpio/binin/binin_changetime” so that when the button is pressed for a duration of less than 500ms, a message “short pressed” is sent to the Serial Monitor. Otherwise, the message “long pressed” is sent to the Serial Monitor.

- Compile and upload your program to ESP32C3
- Video record the output from Serial Monitor and the button pressing action to demonstrate the above logic, and upload to the submission page
- Save and compress your sketch (in zip format), and upload to the submission page